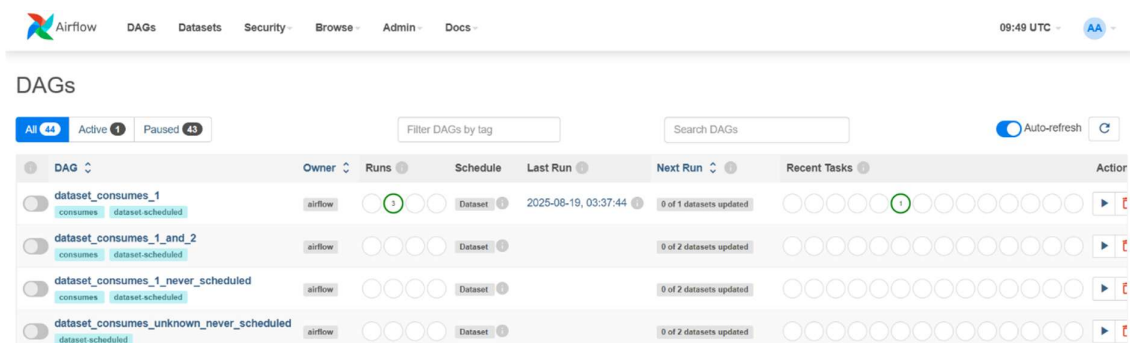


Documentation: Creating a Pipeline in Apache Airflow

This document provides a step-by-step guide to creating and running a pipeline in Apache Airflow.

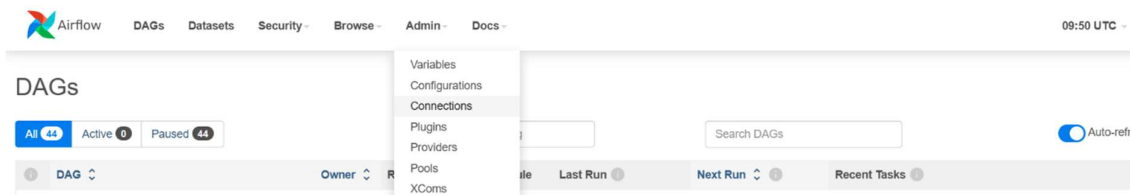
Step 1: Open Airflow Web UI

- Open your browser and navigate to:
 <http://localhost:8080>
- The Airflow home page will open.
- Click on **Admin** in the top navigation bar.



Step 2: Open Connections

- Under the **Admin** menu, click on **Connections**.

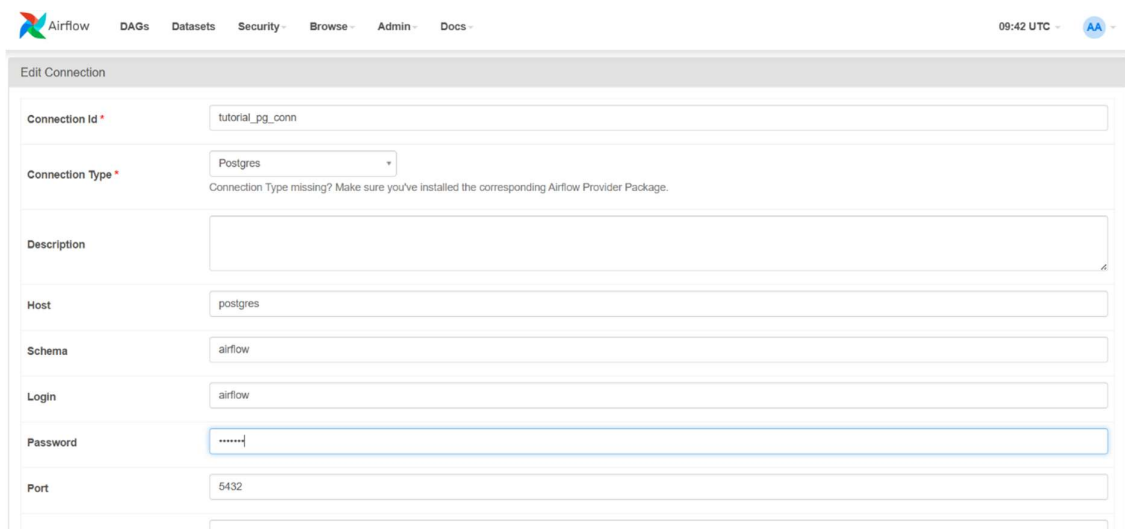


Step 3: Add New Postgres Connection

- Click the **+** (**Add a new record**) button.
- Fill in the details as below:

Field	Value
Connection ID	tutorial_pg_conn
Connection Type	Postgres
Host	postgres
Database	airflow (default in container)
Login	airflow
Password	airflow
Port	5432

- Click **Save**.



Airflow DAGs Datasets Security Browse Admin Docs 09:42 UTC AA

Edit Connection

Connection Id * tutorial_pg_conn

Connection Type * Postgres
Connection Type missing? Make sure you've installed the corresponding Airflow Provider Package.

Description

Host postgres

Schema airflow

Login airflow

Password

Port 5432

Step 4: Connection Added

- The new connection tutorial_pg_conn is now listed under **Connections**.



List Connection

Search

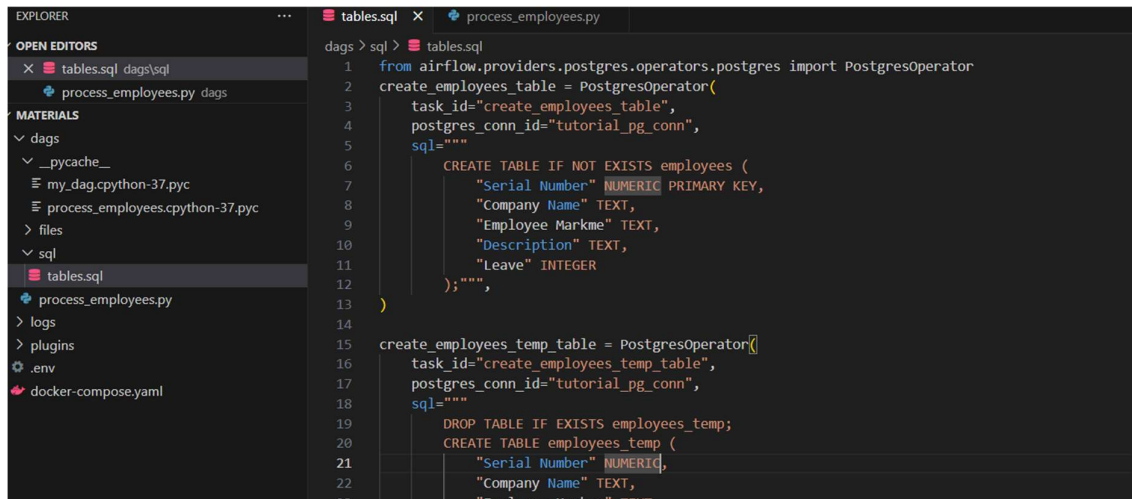
Actions

Record Count: 1

	Conn Id	Conn Type	Description	Host	Port	Is Encrypted	Is Extra Encrypted
☐	tutorial_pg_conn	postgres		postgres	5432	False	False

Step 5: Create SQL File

- Go to your project folder → dags/sql/.
- Create a file named **tables.sql**.
- This file will contain SQL queries to create required tables.

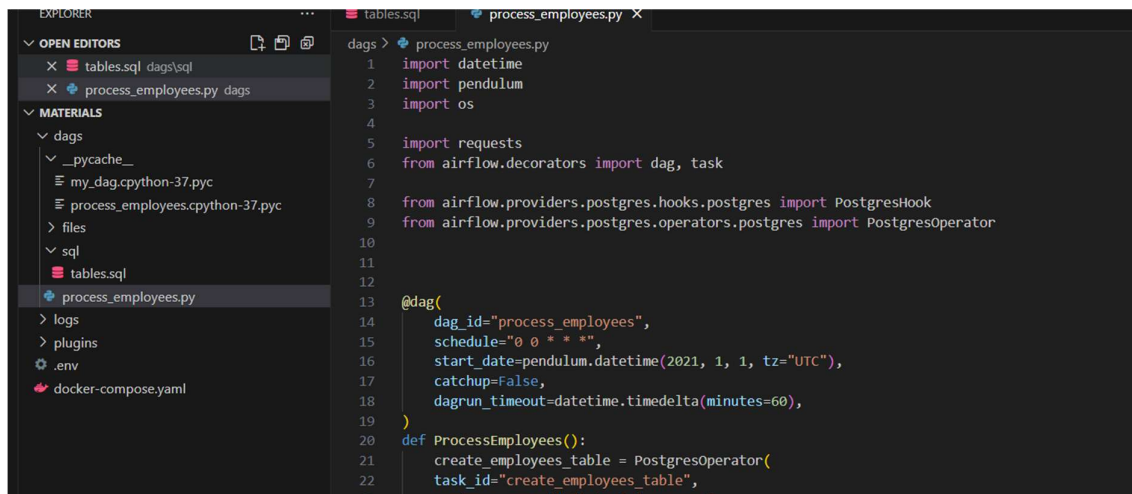


```
EXPLORER
OPEN EDITORS
  tables.sql dags/sql
  process_employees.py dags
MATERIALS
  dags
    __pycache__
    my_dag.cpython-37.pyc
    process_employees.cpython-37.pyc
  files
  sql
    tables.sql
  process_employees.py
  logs
  plugins
  .env
  docker-compose.yaml

dags > sql > tables.sql
1 from airflow.providers.postgres.operators.postgres import PostgresOperator
2 create_employees_table = PostgresOperator(
3     task_id="create_employees_table",
4     postgres_conn_id="tutorial_pg_conn",
5     sql="""
6     CREATE TABLE IF NOT EXISTS employees (
7         "Serial Number" NUMERIC PRIMARY KEY,
8         "Company Name" TEXT,
9         "Employee Markme" TEXT,
10        "Description" TEXT,
11        "Leave" INTEGER
12    );""",
13 )
14
15 create_employees_temp_table = PostgresOperator(
16     task_id="create_employees_temp_table",
17     postgres_conn_id="tutorial_pg_conn",
18     sql="""
19     DROP TABLE IF EXISTS employees_temp;
20     CREATE TABLE employees_temp (
21         "Serial Number" NUMERIC,
22         "Company Name" TEXT,
23         "Employee Markme" TEXT
```

Step 6: Create Python DAG File

- Go to your project folder → dags/.
- Create a file named **process_employee.py**.
- This file defines the pipeline (DAG) for processing employees.

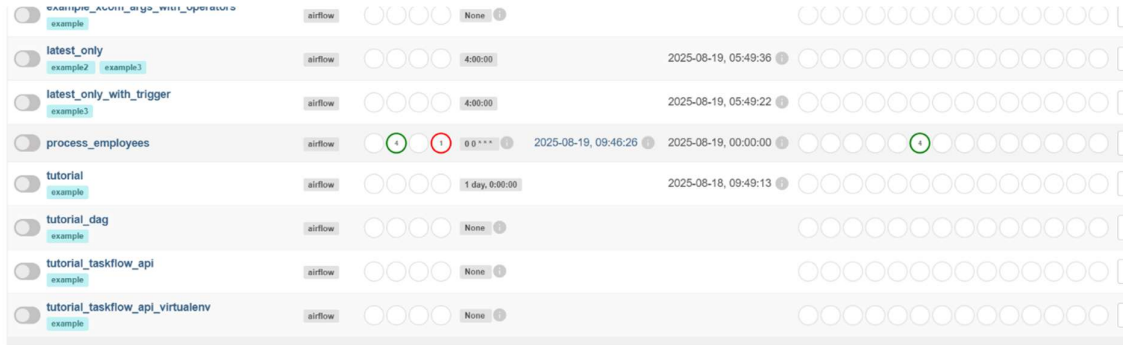


```
EXPLORER
OPEN EDITORS
  tables.sql dags/sql
  process_employees.py dags
MATERIALS
  dags
    __pycache__
    my_dag.cpython-37.pyc
    process_employees.cpython-37.pyc
  files
  sql
    tables.sql
  process_employees.py
  logs
  plugins
  .env
  docker-compose.yaml

dags > process_employees.py
1 import datetime
2 import pendulum
3 import os
4
5 import requests
6 from airflow.decorators import dag, task
7
8 from airflow.providers.postgres.hooks.postgres import PostgresHook
9 from airflow.providers.postgres.operators.postgres import PostgresOperator
10
11
12
13 @dag(
14     dag_id="process_employees",
15     schedule="0 0 * * *",
16     start_date=pendulum.datetime(2021, 1, 1, tz="UTC"),
17     catchup=False,
18     dagrun_timeout=datetime.timedelta(minutes=60),
19 )
20 def ProcessEmployees():
21     create_employees_table = PostgresOperator(
22         task_id="create_employees_table",
23         postgres_conn_id="tutorial_pg_conn"
```

Step 7: DAG Created

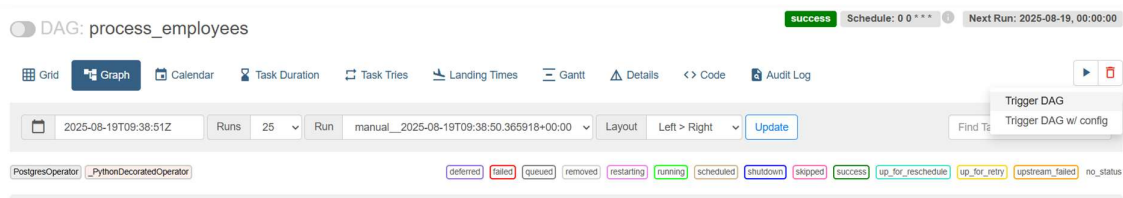
- Open the Airflow UI (localhost:8080).
- In the **DAGs list**, you will now see **process_employee** listed.



DAG Name	Provider	Next Run	Status
example	airflow	None	None
latest_only	airflow	2025-08-19, 05:49:36	None
latest_only_with_trigger	airflow	2025-08-19, 05:49:22	None
process_employees	airflow	2025-08-19, 00:00:00	Success
tutorial	airflow	2025-08-18, 09:49:13	None
tutorial_dag	airflow	None	None
tutorial_taskflow_api	airflow	None	None
tutorial_taskflow_api_virtualenv	airflow	None	None

Step 8: Trigger the DAG

- On the right side of the **process_employee DAG**, click on the **Trigger DAG** button.



DAG: process_employees

Grid Graph Calendar Task Duration Task Tries Landing Times Gantt Details Code Audit Log

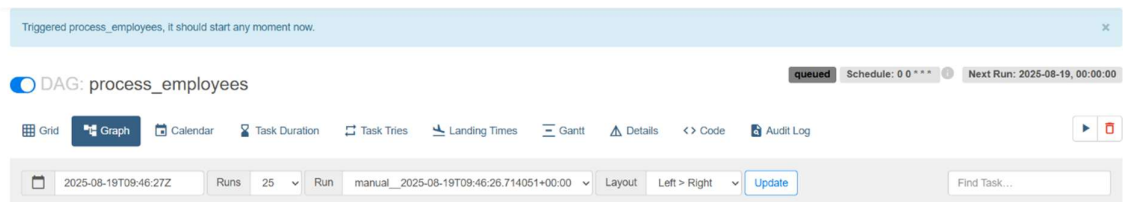
2025-08-19T09:38:51Z Runs 25 Run manual__2025-08-19T09:38:50.365918+00:00 Layout Left > Right Update

PostgresOperator _PythonDecoratedOperator

Trigger DAG Trigger DAG w/ config

Step 9: DAG Started

- The DAG will start running.
- You can see the status of each task in the **Graph view** or **Tree view**.



Triggered process_employees, it should start any moment now.

DAG: process_employees

Grid Graph Calendar Task Duration Task Tries Landing Times Gantt Details Code Audit Log

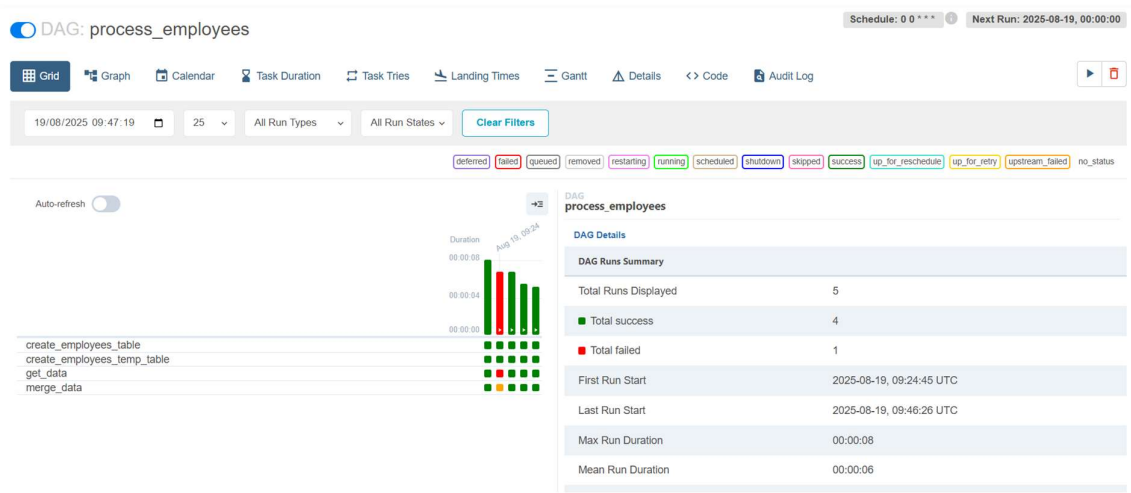
2025-08-19T09:46:27Z Runs 25 Run manual__2025-08-19T09:46:26.714051+00:00 Layout Left > Right Update

Find Task...

Step 10: Explore DAG

- Click on the DAG to explore details.
- You can check **Logs**, **Graph**, **Tree**, and **Task Instances** for more information.

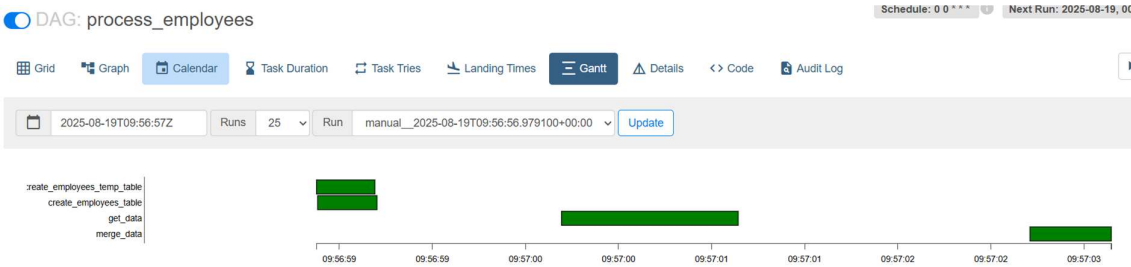
Grid View:



Graph View:



Gantt View:



DAG Details:

DAG: process_employees

Schedule: 0 0

Grid

Graph

Calendar

Task Duration

Task Tries

Landing Times

Gantt

Details

<> Code

Audit Log

DAG Details

failed 1

success 22

upstream_failed 1

Schedule Interval	0 0 * * *
Catchup	False
Started	True
End Date	None
Max Active Runs	0 / 16
Concurrency	16
Default Args	{}
Tasks Count	4
Task IDs	['create_employees_table', 'create_employees_temp_table', 'get_data', 'merge_data']
Relative file location	process_employees.py
Owner	airflow

Successfully created and executed a pipeline in Apache Airflow.
